

SCENERY: a lightweight block cipher based on Feistel structure

Jingya FENG^{1,2}, Lang LI (✉)^{1,2,3}

¹ Hunan Provincial Key Laboratory of Intelligent Information Processing and Application,
Hengyang Normal University, Hengyang 421002, China

² College of Information Science and Engineering, Hunan Normal University, Changsha 410081, China

³ College of Computer Science and Technology, Hengyang Normal University, Hengyang 421002, China

© Higher Education Press 2022

Abstract In this paper, we propose a new lightweight block cipher called SCENERY. The main purpose of SCENERY design applies to hardware and software platforms. SCENERY is a 64-bit block cipher supporting 80-bit keys, and its data processing consists of 28 rounds. The round function of SCENERY consists of $8 \times 4 \times 4$ S-boxes in parallel and a 32×32 binary matrix, and we can implement SCENERY with some basic logic instructions. The hardware implementation of SCENERY only requires 1438 GE based on 0.18 μm CMOS technology, and the software implementation of encrypting or decrypting a block takes approximately 1516 clock cycles on 8-bit microcontrollers and 364 clock cycles on 64-bit processors. Compared with other encryption algorithms, the performance of SCENERY is well balanced for both hardware and software. By the security analyses, SCENERY can achieve enough security margin against known attacks, such as differential cryptanalysis, linear cryptanalysis, impossible differential cryptanalysis and related-key attacks.

Keywords lightweight block cipher, feistel structure, cryptanalysis, internet of things

1 Introduction

With the advent of 5G, high-speed bandwidth and more rapid computing capabilities, the Internet of Things (IoT) has a good development environment. At the same time, lightweight encryption technology is urgently needed to protect sensitive data in the IoT, because embedded devices used in IoT have limited computing resources. In recent years, lightweight block ciphers designed with various strategies have been proposed one after another, such as SFN [1], RECTANGLE [2], SIMON and SPECK [3], GIFT [4], TWINE [5], LBlock [6], KLEIN [7], and PRESENT [8].

In the current IoT era, software and hardware are inseparable. Therefore, we propose a new lightweight block cipher. The main design idea of SCENERY is to design an F function suitable for hardware and software platforms by using bit-slice techniques. RECTANGLE using bit-slice techniques is a

lightweight block cipher with a similar motivation. Bit-slice technique was proposed by E. Biham in 1997 [9]. The bit-slice techniques can not only improve the performance of software, but also resist the cache time attacks and reduce the cost of protected channel attacks.

For the purpose of the trade-offs between safety, cost and efficiency, we employ a balanced Feistel structure. The disadvantage of the balanced Feistel structure is that the diffusion performance is poor, which will result in slower speed. In order to overcome the problem, we adopt a 32×32 binary matrix that can be implemented by simple logic operations, which not only improves the diffused speed of the round function, but also guarantees the algorithm to produce balanced performance on software and hardware implementation. As a result, SCENERY can make efficient use of (embedded) software and also can make a compact unification of encryption and decryption. In order to solve the problem that the master key in the algorithm can be pushed backward, we propose a new key scheduling method. Although this method does not completely solve this problem, it makes the reverse derivation more difficult than the PRESENT's key scheduling, and improves the security of key expansion. Our proposal involves (1) balanced Feistel, (2) S-boxes, and (3) binary matrices. As far as we know, this is the first attempt to use bit-slice techniques in a Feistel structure. The S-boxes are the same as the S-boxes of RECTANGLE, but SCENERY is an independent work and has several concrete design advantages (See Section 3).

We implemented SCENERY on hardware and software. For the hardware implementation, using SMIC 0.18 μm standard cell at 100 KHz, the round-based implementation can get a throughput of 228 Kbits/sec with 1438 GE. For the software implementation, due to the use of bit-slice techniques, the round function can be implemented using a sequence of 32 basic logical instructions. On a 2.3 GHz Intel(R) Core i5-8300H CPU, for one block data, the bit-slice implementation reaches a speed of about 45.5 cycles/byte for encryption and decryption. The speed of software implementation is not the fastest, but the performance of SCENERY is well balanced for hardware and software.

2 Specification of SCENERY

SCENERY is a 64-bit block cipher supporting 80-bit keys, and its data processing consists of 28 rounds. The round function of SCENERY consists of 8 4×4 S-boxes in parallel and a 32×32 binary matrix. We will introduce the encryption process, the decryption process, and the key schedule in details.

2.1 Notations

In the specification of SCENERY, we use the following notations:

2.2 Encryption process

The encryption of SCENERY consists of a Nr -round iterative structure which is based on the balanced Feistel. The encryption of SCENERY is illustrated in Fig. 1. The inputs of the encryption are a 64-bit plaintext denoted by $P \in \{0, 1\}^{64}$ and an 80-bit key denoted by $K \in \{0, 1\}^{80}$, and the output is a 64-bit ciphertext denoted by $C \in \{0, 1\}^{64}$. The encryption procedure is presented as in Table 2.

The components used in SCENERY are as follows.

Round function: The round function of SCENERY is defined as follows: ($1 \leq i \leq 28$)

$$R^{i+1}_{(32)} = L^i_{(32)},$$

$$L^{i+1}_{(32)} = R^i_{(32)} \oplus F(L^i_{(32)}, SK^i_{(32)}).$$

Table 1 Notations of SCENERY

Notations	Descriptions
P	64-bit plaintext
C	64-bit ciphertext
K	80-bit master key or round key
L^i, R^i	32-bit L and R in the i round
SK^i	32-bit round sub-key SK in the i round
F	function F
Nr	round number
S	4×4 S-box
Mix	MixColumns
$\parallel$	concatenation of two binary strings
$\oplus$	bitwise exclusive-OR operation
$\lll$	left circular shift operation

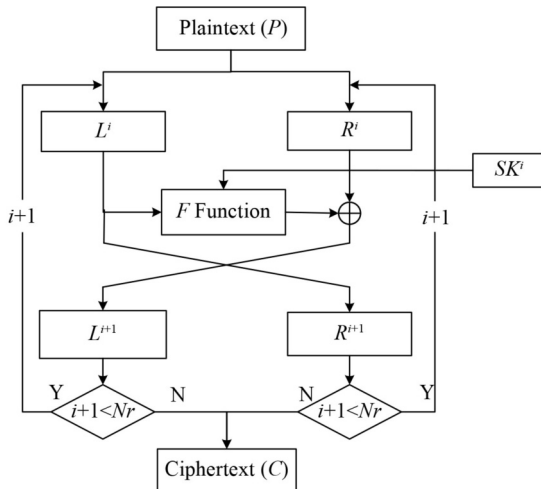


Fig. 1 The encryption of SCENERY

Table 2 The encryption routine of SCENERY

Algorithm 1: SCENERY encryption routine
Input: $P_{(64)}, K_{(80)}$
Output: $C_{(64)}$
1: $P_{(64)} \rightarrow P_{0(8)} \parallel P_{1(8)} \parallel P_{2(8)} \parallel P_{3(8)} \parallel P_{4(8)} \parallel P_{5(8)} \parallel P_{6(8)} \parallel P_{7(8)}$
2: $P_{0(8)} \parallel P_{1(8)} \parallel P_{2(8)} \parallel P_{3(8)} \rightarrow L^1_{(32)}$,
$P_{4(8)} \parallel P_{5(8)} \parallel P_{6(8)} \parallel P_{7(8)} \rightarrow R^1_{(32)}$
3: GenrateKey ($K_{(80)}, SK$)
4: for $i=1$ to Nr do the following
5: $R^{i+1}_{(32)} \leftarrow L^i_{(32)}$,
$L^{i+1}_{(32)} \leftarrow R^i_{(32)} \oplus F(L^i_{(32)}, SK^i_{(32)})$
6: end for
7: $P_{0(8)} \parallel P_{1(8)} \parallel P_{2(8)} \parallel P_{3(8)} \leftarrow R^i_{(32)}$,
$P_{4(8)} \parallel P_{5(8)} \parallel P_{6(8)} \parallel P_{7(8)} \leftarrow L^i_{(32)}$
8: $C_{(64)} \leftarrow P_{(64)}$

The function F consists of the following three steps: AddRoundKey, SubColumns, and MixColumns. The initial value of $L^1 \parallel R^1$ in the first round is the 64-bit plaintext, and the value of $SK^1_{(32)}$ in the first round is $K_0 \parallel K_1 \parallel K_2 \parallel K_3$. 32-bit L^i and R^i are divided into four groups and the length of each group is eight bits.

AddRoundKey: The operation of the AddRoundKey combines 32-bit L^i with 32-bit SK^i , respecting array positioning, using bitwise exclusive-or.

SubColumns: The operation of the SubColumns is column-based permutation, and Fig. 2 illustrates the SubColumns in detail. The S-boxes are the same as the S-boxes of RECTANGLE, as shown in Table 3.

MixColumns: SCENERY uses a 32×32 binary matrix defined as follows:

$$M = \begin{bmatrix} M_0 & 0 \\ 0 & M_1 \end{bmatrix}.$$

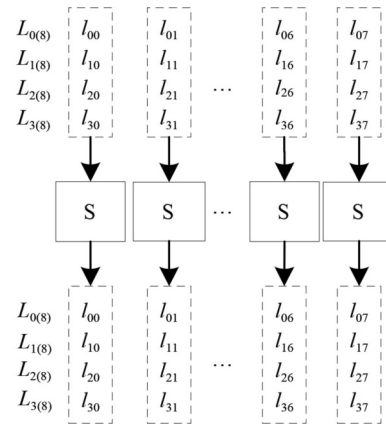


Fig. 2 The SubColumns of SCENERY

Table 3 4-bit S-box in hexadecimal form

x	$S[x]$	x	$S[x]$	x	$S[x]$	x	$S[x]$
0	6	4	1	8	B	C	8
1	5	5	E	9	0	D	F
2	C	6	7	A	3	E	4
3	A	7	9	B	D	F	2

$$M_0 = \begin{bmatrix} 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_1 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

Fig. 3 The M_0 and M_1 of SCENERY

M_0 and M_1 are 16×16 binary matrices (see Fig. 3).

2.3 Decryption process

The decryption process of SCENERY is the same as the encryption process. But the round sub-keys in the decryption algorithm are used in a reverse order, e.g., the sub-key used in the first round of decryption is the sub-key used in the last round of encryption.

2.4 Key schedule

The 80-bit key is denoted as $K = k_0, k_1, k_2, \dots, k_{78}, k_{79}$ and divided into 8-bit K_j ($0 \leq j \leq 79$):

$$K_0 = k_0 \parallel k_1 \dots k_6 \parallel k_7; \quad K_1 = k_8 \parallel k_9 \dots k_{14} \parallel k_{15};$$

$$K_2 = k_{16} \parallel k_{17} \dots k_{22} \parallel k_{23}; \quad K_3 = k_{24} \parallel k_{25} \dots k_{30} \parallel k_{31};$$

$$K_4 = k_{32} \parallel k_{33} \dots k_{39} \parallel k_{40}; \quad K_5 = k_{40} \parallel k_{41} \dots k_{46} \parallel k_{47};$$

$$K_6 = k_{48} \parallel k_{49} \dots k_{54} \parallel k_{55}; \quad K_7 = k_{56} \parallel k_{57} \dots k_{62} \parallel k_{63};$$

$$K_8 = k_{64} \parallel k_{65} \dots k_{70} \parallel k_{71}; \quad K_9 = k_{72} \parallel k_{73} \dots k_{78} \parallel k_{79}.$$

The 32-bit round sub-key SK^i is the first four bytes of the 80-bit round key K^i and denoted as $SK^i = K_0^i, K_1^i, K_2^i, K_3^i$. The key schedule consists of SubCells, left cyclic shift, round constant addition and dynamic permutation (DP). The four steps are as follows:

1) Replace the 8-bit data in the input key K^i (the 80-bit K in the i round) with the S-boxes in Table 4:

$$(k_{12}^i k_{13}^i k_{14}^i k_{15}^i) \leftarrow S(k_{12}^i k_{13}^i k_{14}^i k_{15}^i);$$

$$(k_{28}^i k_{29}^i k_{30}^i k_{31}^i) \leftarrow S(k_{28}^i k_{29}^i k_{30}^i k_{31}^i).$$

2) The result of step 1) is rotated 11 bits to the left:

$$(k_0^{i+1} k_1^{i+1} \dots k_{78}^{i+1} k_{79}^{i+1}) \leftarrow (k_{11}^i k_{12}^i \dots k_9^i k_{10}^i);$$

3) The round constant is the round number i . The round constant addition is as follows:

$$(k_{11}^{i+1} k_{12}^{i+1} k_{13}^{i+1} k_{14}^{i+1} k_{15}^{i+1}) \leftarrow$$

$$(k_{11}^{i+1} k_{12}^{i+1} k_{13}^{i+1} k_{14}^{i+1} k_{15}^{i+1}) \oplus (i_4 i_3 i_2 i_1 i_0);$$

Table 4 Results on different rounds of SCENERY

Rounds	c	Avg. AE	Avg. SAC
4	1	31.9030	0.9997
5	1	32.0014	0.9997
6	1	32.0015	0.9996
7	1	32.0006	0.9997
28	1	32.0011	0.9996

“c” is the completeness, “Avg.” represents the average of the test results based on the test cases.

4) Dynamic Permutation (DP): Get the next key K^{i+1} according to the value of $v_0(k_{14}^i k_{15}^i)$ in the input key K^i and the result of step 3). The value of K^{i+1} is shown as follows:

$$K^{i+1} = K_{(v_0 \cdot 2) \% 10}^{i+1} \parallel K_{(v_0 \cdot 2 + 1) \% 10}^{i+1} \parallel K_{(v_0 \cdot 2 + 2) \% 10}^{i+1} \dots$$

$$K_{(v_0 \cdot 2 + 7) \% 10}^{i+1} \parallel K_{(v_0 \cdot 2 + 8) \% 10}^{i+1} \parallel K_{(v_0 \cdot 2 + 9) \% 10}^{i+1};$$

3 Design rationale

3.1 The structure of the SCENERY

The main structures used by lightweight block ciphers are Feistel, SPN and generalized Feistel, and each structure has its own advantages. The Feistel structure is highly symmetric, and the decryption reuses the encryption module, which consumes less hardware resources during implementation. However, its round function only changes half of the block message in a round of iteration, which results in slow diffusion of cryptographic algorithms. The SPN structure spreads fast, but the symmetry is relatively poor, and it consumes more resources during the implementation of the encryption and decryption. Although some SPN-based algorithms, by designing special components (such as involutive matrices and involutive S-boxes), the reuse rate of components can be improved and resources can be reduced, security needs further study. The generalized Feistel structure is highly symmetrical, and the decryption reuses the encryption module, which consumes less hardware resources during implementation, but it must perform more rounds to provide security. Considering the compromise of three aspects: security, cost, and efficiency, Feistel structure seems to be the best choice.

3.2 Linear layer

The linear layer of block ciphers generally implements diffusion through some components such as bit permutations, ShiftRows, and MixColumns. The linear layer of SCENERY consists of one component: MixColumns.

As we all know, the number of branches in matrix can be used to measure the security of the diffusion layer. The number of branches affects the ability of block ciphers to resist analysis. Therefore, using the matrix in diffusion layers can increase the diffusion speed and improve the security. Considering the safe and fast implementation using bit-slice techniques, we chose a 32×32 binary matrix M with four branches.

Lemma 1 Let matrix M be the generator matrix for an F2-linear $(2n, 2^n)$ code with minimal distanced such that the dual code has minimum distance d as well. Then M has at least $d-1$ ones per row and per column.

According to the block idea of matrices, the matrix M is decomposed into four 16×16 matrices (M_0, M_1 and two zero matrices). From Lemma 1, we can know M with d branches if and only if all $(n-d+1) \times k$ block submatrices of M have full rank for $0 \leq k < d-1$. So, M_0 and M_1 should provide four branches, that is, M_0 and M_1 have three ones per row and column.

In order to use bit-slice techniques to achieve fast and secure diffusion, we use two 8-bit binary data vectors a and b to construct a 16×16 binary matrix with four branches. Assuming that the hamming weight of a is one and the

hamming weight of b is two, the possible values of a and b can be derived.

Considering the area consumption, the vectors a and b that satisfied the condition of $b = a \wedge c$ (where the hamming weight of c is one) are paired. Then, construct a 16×16 binary matrix with four branches using the paired results (a, b) and (b, a) . Finally, test and choose the most suitable matrix.

Let $L_0^i, L_1^i, L_2^i, L_3^i$ be four 8-bit inputs of the MixColumns, $L_0^{i+1}, L_1^{i+1}, L_2^{i+1}, L_3^{i+1}$ be the four 8-bit outputs. The MixColumns can be implemented as follows:

$$\begin{cases} t = L_1^i \lll 1 \oplus L_0^i \ggg 3; \\ L_0^{i+1} = L_0^i \ggg 2 \oplus t; \\ L_1^{i+1} = L_1^i \oplus t; \end{cases} \begin{cases} z = L_3^i \lll 4 \oplus L_2^i \lll 1; \\ L_2^{i+1} = L_2^i \oplus z; \\ L_3^{i+1} = L_3^i \ggg 3 \oplus z; \end{cases}$$

3.3 Nonlinear layer

The operating environment of lightweight block ciphers is a low-cost product with relatively few computing resources, so SCENERY cannot be too complicated, and the S-boxes used cannot be too large. Furthermore, the S-boxes are the only non-linear component of cryptographic algorithms, and the security of SCENERY largely depends on the security of the S-boxes. Considering the balance between security and efficiency, SCENERY uses 4×4 S-boxes.

The non-linear layer of PRESENT realized confusion, and satisfied the low power consumption and 4×4 small-scale design concepts. But the different S-boxes are used for the encryption and decryption, which affects the efficiency of the hardware implementation. Moreover, the permutation based on bits is not conducive to the implementation of software. In order to further improve the efficiency of hardware implementation, a special class of S-boxes has received the attention of designers - reflexive S-boxes. Many lightweight block ciphers have adopted involutive S-boxes, such as KLEIN. However, it is necessary to further discuss whether there is a hidden danger in this particular feature of invertibility. Meanwhile, [6] proposed LBlock based on a balanced Feistel structure. The linear operations of LBlock include a left circular shift 8 bits and a 4-bit replacement which has better hardware performance, and also has higher efficiency on 8-bit microcontroller. But LBlock has eight different 4×4 S-boxes. In order to design an algorithm suitable for multiple platforms, RECTANGLE was proposed in 2015. The 4×4 S-boxes of RECTANGLE use bit-slice techniques and can be implemented using 12 basic logic instructions, which is conducive to software and hardware implementation.

Considering the performance of hardware and software implementation, the S-boxes of SCENERY are the same as the S-boxes of RECTANGLE.

3.4 Key schedule

Our main goal is to minimize cost and ensure security. To improve security, we use two S-boxes to implement the non-linear permutation of key expansion. And we choose a left cyclic shift to provide appropriate diffusion, which does not require hardware area. For the round constants, we directly use rounds as the round constants.

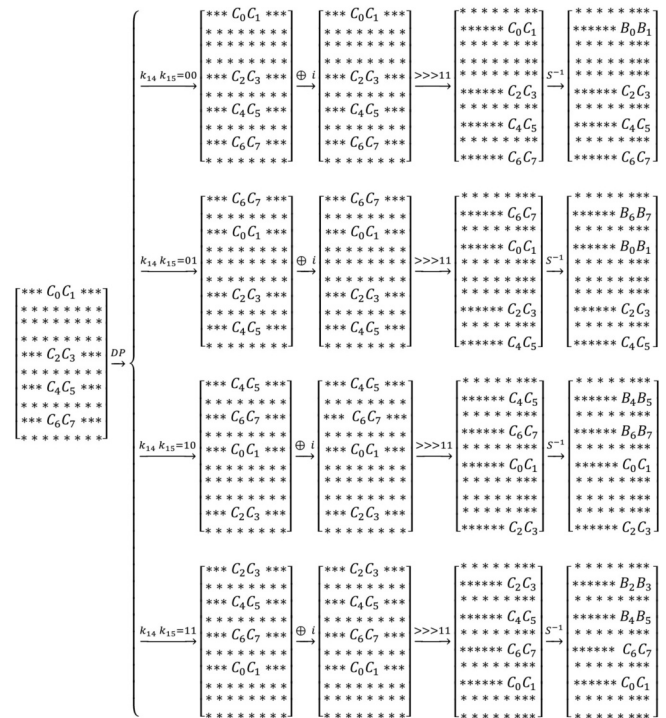
For DP, [10] propose a new key expansion method to solve the problem that the round key of AES is a simple iteration from the last round key. For lightweight block ciphers, there is also the same problem. But the method in [10] is not suitable for SCENERY. Faced with this problem, we propose a new method, which uses 2-bit in the round key as the control signal, and dynamically replaces the intermediate result of key schedule to obtain the next round key.

Compared with the PRESENT's key schedule, we can not get the last round key directly. For the round key in the $(i+1)$ round, it does not know how the intermediate result is replaced.

Assume that the 80-bit round key in the $(i+1)$ round is:

$$\begin{bmatrix} * & * & * & C_0 & C_1 & * & * & * \\ * & * & * & * & * & * & * & * \\ * & * & * & * & * & * & * & * \\ * & * & * & * & * & * & * & * \\ * & * & * & C_2 & C_3 & * & * & * \\ * & * & * & * & * & * & * & * \\ * & * & * & C_4 & C_5 & * & * & * \\ * & * & * & * & * & * & * & * \\ * & * & * & C_6 & C_7 & * & * & * \\ * & * & * & * & * & * & * & * \end{bmatrix}$$

We need to make assumptions about the four possible permutations and judge the results of the inverse for the four assumptions. This process is shown as follows:



Based on the four results above, we need to judge whether the values of B_0B_1 , B_2B_3 , B_4B_5 , and B_6B_7 are equal to the values we assumed. We know that the probability of the values (00, 01, 10 and 11) is 2^{-2} .

When the value of the round key in the $(i+1)$ round is the correct value, there are two possibilities of the round key in

the i round:

1) only one of the results satisfies its assumption, we can get the value of the round key in the i round, and the probability is $2^{-0.7}$.

2) we may get 2-4 possible values of the round key in the i round, and the probability is $2^{-1.1}$.

When the value of the round key in the $(i+1)$ round is a possible value, there are three possibilities of the round key in the i round:

1) only one of the results satisfies its assumption, we can get the value of the round key in the i round, and the probability is $4 \times (1/4) \times ((3/4)^3)$.

2) we may get 2-4 possible values of the round key in the i round, and the probability is $2^{-1.9}$.

3) The value of the round key in the i round does not exist, and the probability is $(3/4)^4$.

We only analyzed one round of the reverse derivation, and multiple rounds of the reverse derivation will be more complicated. Although this method does not completely solve this problem, it makes the reverse derivation more difficult than the PRESENT's key schedule, and improves the security of key expansion.

4 Security evaluation

4.1 Dependency and avalanche effect

A good avalanche effect is one of the most important cryptographic design goals. Kam and Davida [11] and H. Feistel [12] first proposed the concepts of the dependency and the avalanche effect (referred to as AE). Afterwards, Webster and Tavares [13] introduced the strict avalanche criterion (referred to as SAC). By studying [14], we tested the dependency, the completeness, the avalanche effect, and the strict avalanche criterion of SCENERY.

We selected 10000 specific plaintext blocks and randomly changed one bit, two bits, three bits, four bits to form some test cases. Based on these test cases, we tested the completeness, the avalanche effect and the strict avalanche criterion of different rounds. The results are shown in Table 4. If the input changes slightly (for example, flipping one bit), the output changes significantly (for example, more than half of the output bits flip), the avalanche effect is considered safer. Figures. 4 and 5 show the avalanche effects of a specific plaintext and a specific key. According to the results, we can know that SCENERY reaches full dependency after four rounds, so we believe that SCENERY is sufficient to resist differential cryptanalysis.

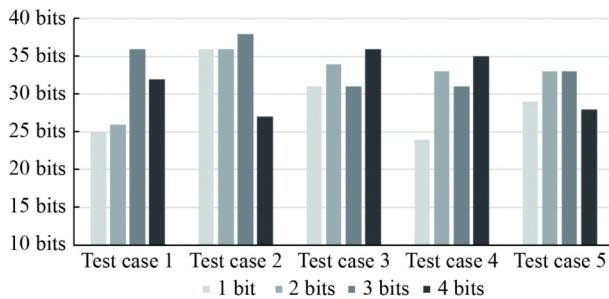


Fig. 4 Analysis of avalanche effect of a specific plaintext

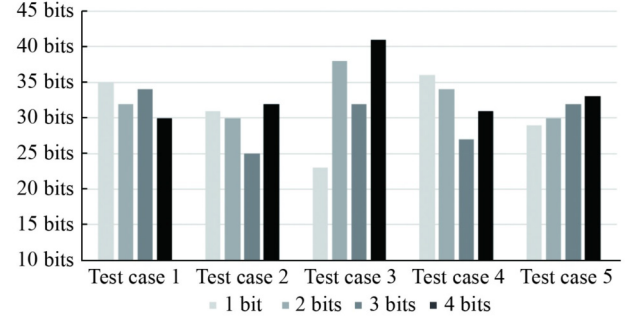


Fig. 5 Analysis of avalanche effect of a specific key

4.2 Differential cryptanalysis

For differential cryptanalysis (DC) and Linear cryptanalysis (LC), from [15], we can know estimation of block ciphers against differential and linear cryptanalysis ability is the key to calculating the number of active S-boxes. We construct a bit-oriented Mixed Integer Linear Programming (MILP) model using the method in [16] to calculate the number of active S-boxes. Each differential variable x is regarded as 32-bit ΔL , and each differential variable y is regarded as 32-bit ΔR . The round function of SCENERY is expressed as follows:

$$\begin{aligned}
 \Delta L &= \begin{bmatrix} x_0 & x_1 & x_2 & x_3 & x_4 & x_5 & x_6 & x_7 \\ x_8 & x_9 & x_{10} & x_{11} & x_{12} & x_{13} & x_{14} & x_{15} \\ x_{16} & x_{17} & x_{18} & x_{19} & x_{20} & x_{21} & x_{22} & x_{23} \\ x_{24} & x_{25} & x_{26} & x_{27} & x_{28} & x_{29} & x_{30} & x_{31} \end{bmatrix} \\
 \xrightarrow{SC} & \begin{bmatrix} s_0 & s_1 & s_2 & s_3 & s_4 & s_5 & s_6 & s_7 \\ s_8 & s_9 & s_{10} & s_{11} & s_{12} & s_{13} & s_{14} & s_{15} \\ s_{16} & s_{17} & s_{18} & s_{19} & s_{20} & s_{21} & s_{22} & s_{23} \\ s_{24} & s_{25} & s_{26} & s_{27} & s_{28} & s_{29} & s_{30} & s_{31} \end{bmatrix} \\
 \xrightarrow{Mix} & \begin{bmatrix} m_0 & m_1 & m_2 & m_3 & m_4 & m_5 & m_6 & m_7 \\ m_8 & m_9 & m_{10} & m_{11} & m_{12} & m_{13} & m_{14} & m_{15} \\ m_{16} & m_{17} & m_{18} & m_{19} & m_{20} & m_{21} & m_{22} & m_{23} \\ m_{24} & m_{25} & m_{26} & m_{27} & m_{28} & m_{29} & m_{30} & m_{31} \end{bmatrix} \\
 \oplus \Delta R & \rightarrow \begin{bmatrix} x_{32} & x_{33} & x_{34} & x_{35} & x_{36} & x_{37} & x_{38} & x_{39} \\ x_{40} & x_{41} & x_{42} & x_{43} & x_{44} & x_{45} & x_{46} & x_{47} \\ x_{48} & x_{49} & x_{50} & x_{51} & x_{52} & x_{53} & x_{54} & x_{55} \\ x_{56} & x_{57} & x_{58} & x_{59} & x_{60} & x_{61} & x_{62} & x_{63} \end{bmatrix} \xrightarrow{NextRound} \Delta L \\
 \Delta L & \rightarrow \begin{bmatrix} y_{32} & y_{33} & y_{34} & y_{35} & y_{36} & y_{37} & y_{38} & y_{39} \\ y_{40} & y_{41} & y_{42} & y_{43} & y_{44} & y_{45} & y_{46} & y_{47} \\ y_{48} & y_{49} & y_{50} & y_{51} & y_{52} & y_{53} & y_{54} & y_{55} \\ y_{56} & y_{57} & y_{58} & y_{59} & y_{60} & y_{61} & y_{62} & y_{63} \end{bmatrix} \xrightarrow{NextRound} \Delta R
 \end{aligned}$$

We defined A_i as an S-box. if the input difference of A_i is non-zero, A_i is an active S-box, and the value of A_i is one. Otherwise, the value of A_i is 0. The constraints are as follows:

$$\begin{cases} x_{i+24} + x_{i+16} + x_{i+8} + x_i - A_i \geq 0 \\ A_i - x_i \geq 0 \\ A_i - x_{i+8} \geq 0 \\ A_i - x_{i+16} \geq 0 \\ A_i - x_{i+24} \geq 0 \\ i = (0, 1, 2, 3, 4, 5, 6, 7) \end{cases}$$

For a 4×4 S-box, a non-zero input difference must result in

a non-zero output difference, and vice versa. The inequality constraints are as follows:

$$\left\{ \begin{array}{l} s_i + s_{i+8} + s_{i+16} + s_{i+24} - x_i \geq 0 \\ s_i + s_{i+8} + s_{i+16} + s_{i+24} - x_{i+8} \geq 0 \\ s_i + s_{i+8} + s_{i+16} + s_{i+24} - x_{i+16} \geq 0 \\ s_i + s_{i+8} + s_{i+16} + s_{i+24} - x_{i+24} \geq 0 \\ x_i + x_{i+8} + x_{i+16} + x_{i+24} - s_i \geq 0 \\ x_i + x_{i+8} + x_{i+16} + x_{i+24} - s_{i+8} \geq 0 \\ x_i + x_{i+8} + x_{i+16} + x_{i+24} - s_{i+16} \geq 0 \\ x_i + x_{i+8} + x_{i+16} + x_{i+24} - s_{i+24} \geq 0 \\ i = (0, 1, 2, 3, 4, 5, 6, 7) \end{array} \right.$$

Sun et al. showed that linear inequalities can describe some conditional differential properties of S-boxes [17]. For example, the least significant bit of the output difference of the S-box is always one if the input difference is 1000, that is, $1000 \rightarrow ***1$. The differential properties of the S-boxes are summarized as follows:

$$\left\{ \begin{array}{l} 0001 \rightarrow **1* \\ 0100 \rightarrow **11 \\ 1000 \rightarrow ***1 \\ 0101 \rightarrow **0* \\ 1100 \rightarrow ***0 \end{array} \right\} \left\{ \begin{array}{l} *11* \rightarrow 0010 \\ *1** \rightarrow 0100 \\ *0** \rightarrow 0110 \\ **1* \rightarrow 1100 \\ **0* \rightarrow 1110 \end{array} \right.$$

The following inequality can be obtained, according to the input-output difference characteristics of the S-boxes.

$$\left\{ \begin{array}{l} x_{i+24} + x_{i+16} + x_{i+8} - x_i + s_{i+8} \geq 0 \\ 2x_{i+24} - 2x_{i+16} + 2x_{i+8} + 2x_i + s_{i+8} + s_i \geq 0 \\ -x_{i+24} + x_{i+16} + x_{i+8} + x_i + s_i \geq 0 \\ x_{i+24} - x_{i+16} + x_{i+8} - x_i - s_{i+8} \geq -2 \\ -x_{i+24} - x_{i+16} + x_{i+8} + x_i - s_i \geq -2 \\ 2s_{i+24} + 2s_{i+16} - 2s_{i+8} + 2s_i + x_{i+16} + x_{i+8} \geq 0 \\ s_{i+24} - s_{i+16} + s_{i+8} + s_i + x_{i+16} \geq 0 \\ s_{i+24} - s_{i+16} - s_{i+8} + s_i - x_{i+16} \geq -2 \\ -s_{i+24} - s_{i+16} + s_{i+8} + s_i + x_{i+8} \geq -1 \\ -s_{i+24} - s_{i+16} - s_{i+8} + s_i - x_{i+8} \geq -3 \end{array} \right.$$

Since the MixColumns can be implemented by shift and XOR, it has two 8-bit intermediate variables t and z . Therefore, the MixColumns can be defined as two steps. The first step is the constraints between s , z and t .

$$\left\{ \begin{array}{l} s_{(i+5)\%8} + t_i - s_{8+(i+1)\%8} \geq 0 \\ s_{(i+5)\%8} + s_{8+(i+1)\%8} - t_i \geq 0 \\ s_{8+(i+1)\%8} + t_i - s_{(i+5)\%8} \geq 0 \\ s_{(i+5)\%8} + t_i + s_{8+(i+1)\%8} \leq 2 \\ i = (0, 1, 2, 3, 4, 5, 6, 7) \\ s_{16+(i+1)\%8} + z_i - s_{24+(i+4)\%8} \geq 0 \\ s_{16+(i+1)\%8} + s_{24+(i+4)\%8} - z_i \geq 0 \\ s_{24+(i+4)\%8} + z_i - s_{16+(i+1)\%8} \geq 0 \\ s_{16+(i+1)\%8} + s_{24+(i+4)\%8} + z_i \leq 2 \\ i = (0, 1, 2, 3, 4, 5, 6, 7) \end{array} \right.$$

The second step is the constraints of the output m .

$$\left\{ \begin{array}{l} m_i + t_i - s_{(i+6)\%8} \geq 0 \\ m_i + s_{(i+6)\%8} - t_i \geq 0 \\ s_{(i+6)\%8} + t_i - m_i \geq 0 \\ m_i + t_i + s_{(i+6)\%8} \leq 2 \\ i = (0, 1, 2, 3, 4, 5, 6, 7) \end{array} \right\} \left\{ \begin{array}{l} m_{i+16} + z_i - s_{i+16} \geq 0 \\ m_{i+16} + s_{i+16} - z_i \geq 0 \\ s_{i+16} + z_i - m_{i+16} \geq 0 \\ m_{i+16} + z_i + s_{i+16} \leq 2 \\ m_{i+24} + z_i - s_{24+(i+5)\%8} \geq 0 \\ m_{i+24} + s_{24+(i+5)\%8} - z_i \geq 0 \\ s_{24+(i+5)\%8} + z_i - m_{i+24} \geq 0 \\ m_{i+24} + z_i + s_{24+(i+5)\%8} \leq 2 \\ i = (0, 1, 2, 3, 4, 5, 6, 7) \end{array} \right.$$

The minimum number of differential and linear active S-boxes for n rounds of SCENERY, PRESENT, RECTANGLE, LBlock and GIFT is shown in Table 5. After six rounds, SCENERY has more differential active S-boxes than PRESENT, Twine, RECTANGLE, LBlock and GIFT. We found that there are at least 33 active S-boxes in 11 rounds. Considering that the optimal differential probability of S_i is all equal to 2^{-2} , we can know that the maximum differential probability of 11-round SCENERY satisfies $DCP_{max}^{11r} \leq 2^{33 \times (-2)} = 2^{-66}$. This result means that there is no useful 11-round differential characteristic for SCENERY. Therefore, we believe that the 28-round SCENERY is sufficient to against differential cryptanalysis.

4.3 Linear cryptanalysis

As shown in Table 5, SCENERY has an average of 11 linear active S-boxes every six rounds, but RECTANGLE only has an average of eight linear active S-boxes every six rounds. After five rounds, SCENERY has more linear active S-boxes than other block ciphers. There are at least 32 active S-boxes for 11-round of SCENERY and the maximal linear probability of each S_i is 2^{-2} , the maximal linear probability for 11-round SCENERY satisfies $LCP_{max}^{11r} \leq 2^{32 \times (-2)} = 2^{-64}$. The maximal linear probability of full-round SCENERY is 2^{-150} . Therefore, we believe that the full-round SCENERY is enough secure against linear cryptanalysis.

Table 5 Lower bounds for numbers of differential and linear active S-boxes

Ciphers	D/LC	Rounds																Ref.
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	
PRESENT-80	DC	1	2	4	6	10	12	14	16	18								[4]
	LC										1	2	3	4	5	6	7	
Twine	DC	0	1	2	3	4	6	8	11	14								[5]
	LC										0	1	2	3	4	6	8	
RECTANGLE	DC	1	2	3	4	6	8	11	13	14								[4]
	LC										1	2	3	4	6	8	10	
LBlock	DC	0	1	2	3	4	6	8	11	14								[6]
	LC										0	1	2	3	5	6	8	
GIFT	DC	1	2	3	5	7	10	13	16	18								[4]
	LC										1	2	3	5	7	9	12	
LED	DC	1	5	21	25	26	30	46	50	51								[18]
	LC										1	5	21	25	26	30	46	
Loong	DC	8	16	24	32	40	48	56	64	72								[18]
	LC										8	16	24	32	40	48	56	
SCENERY	DC										0	1	2	4	6	11	15	this work
	LC										0	1	2	4	6	11	14	

4.4 Impossible differential cryptanalysis

We use the MILP-based method proposed by Sasaki, Yu and Y. Todo [19] to find the impossible differential characteristic of SCENERY. The best discriminator is the following 8-round impossible differential characteristics:

$$\begin{pmatrix} 0000 & 0000 & 0000 & 0000 \\ 0000 & 0000 & 0000 & 0000 \\ 0000 & 0001 & 0000 & 0000 \\ 0000 & 0001 & 0000 & 0000 \end{pmatrix} \xrightarrow{8r} \begin{pmatrix} 0000 & 0000 & 0000 & 0000 \\ 0000 & 0000 & 0000 & 0000 \\ 0000 & 0000 & 0000 & 0000 \\ 0000 & 0001 & 0000 & 0000 \end{pmatrix}$$

Since SCENERY reaches the full dependency after four rounds, which is a property that all inputs affect all outputs. We believe that the full-round of SCENERY are sufficient to resist the impossible differential attack.

4.5 Integral cryptanalysis

We found some eight-round higher-order integral distinguishers, using the method in [2]. We first find a four-round integral distinguisher. Choose a set of two plaintexts P and P' , set $p_{55} = p_{39} = 0$, $p'_{55} = p'_{39} = 1$ and set the other 62-bit positions to a fixed random value. The differential probability of the S-box ($0101 \rightarrow **0*$) is 1. Then, the outputs in the round four have a zero difference at position 52, i.e., the XOR sum of the position 52 equals to 0. Next, consider the decryption, and choose a set of 2^{60} plaintexts which have certainly fixed values in 39, 47, 55, 63, while having all the 2^{60} possible values in the other bits. After 4-round encryption, the 2^{60} intermediate values can be divided into 2^{59} subsets, the two

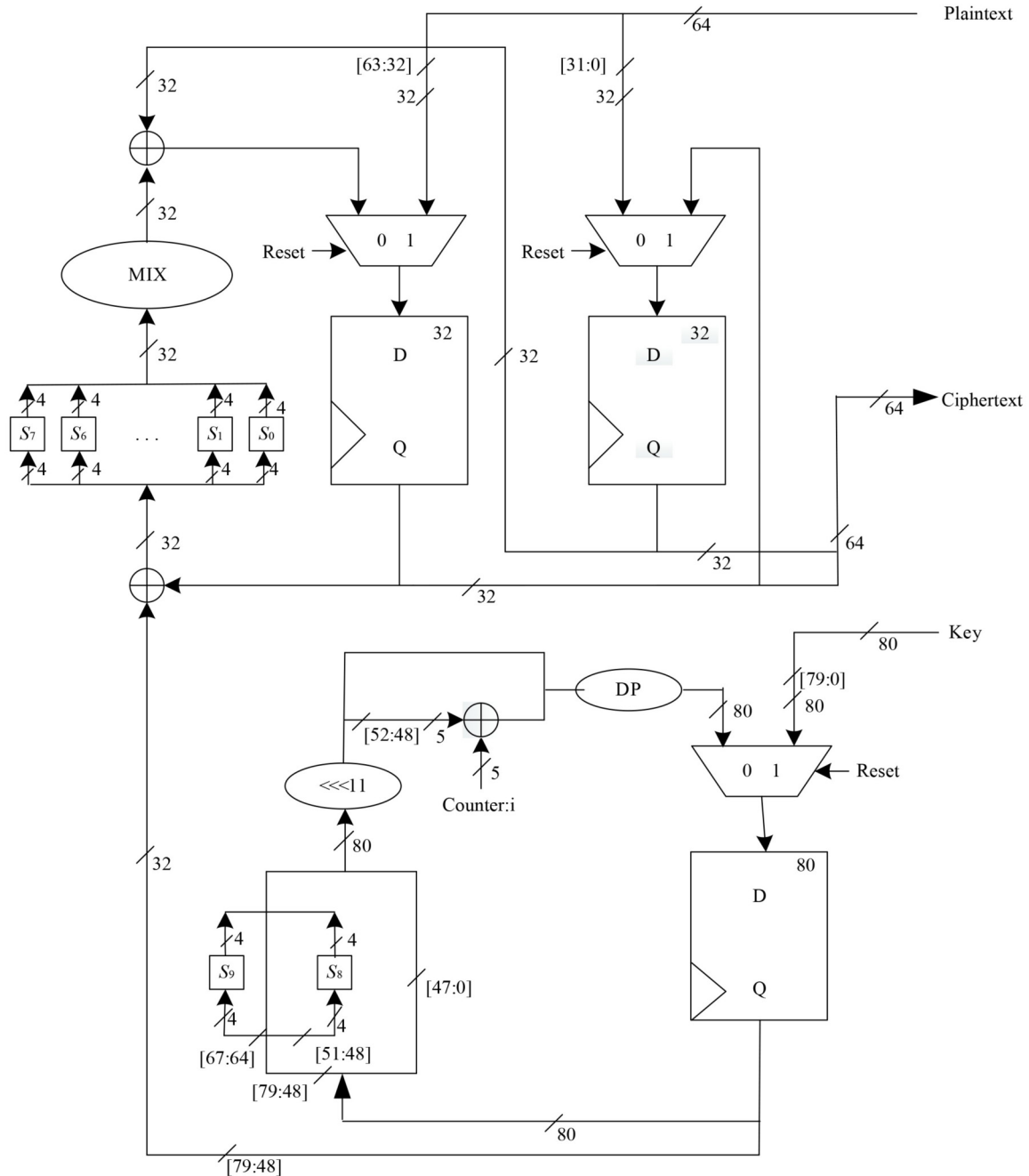


Fig. 6 The datapath of the round-based SCENERY

values in each subset satisfy the data condition of the above four-round distinguisher. Hence, after 8-round encryption, the XOR sum of all the 2^{60} outputs in the position 52 equals to 0, which is an eight-round integral distinguisher. Similarly, we believe that 28-round SCENERY has enough security against integral cryptanalysis.

4.6 Key schedule attacks

For key schedule attacks, the key schedule of SCENERY is similar to the key schedule of the PRESENT. Two S-boxes used for confusion and one left cyclic shift used for diffusion can resist the related-key attack [20]. The round constant addition can effectively prevent the sliding attack [21].

5 Performance evaluation

5.1 Hardware performance

We did the experiment to check the complexity of hardware implementation. SCENERY was implemented in Verilog-HDL and simulated through ModelSim SE 10.1c Evaluation. We synthesized SCENERY for a standard cell library based on SMIC 0.18 μm CMOS technology. All proposed hardware designs were synthesized with Synopsys Design Compiler version B-2008.09.

Figure 6 illustrates the datapath of SCENERY. The round-base SCENERY requires two 32-bit wide registers, one 80-bit wide registers, each is composed of D-flip-flops and requires three 2-to-1 MUX. The cost of a D flip-flop is 4.5 GE, while the cost of a 2 to 1 MUX is 2.0 GE. The 64-bit plaintext of SCENERY is divided into two 32-bit inputs, and stored in the two 32-bit wide registers. The 80-bit key is stored in the 80-bit wide register. The round constant addition, the key addition, the MixColumns and the right data addition are implemented as bit-wise XORs. The S-box layer consists of eight 4×4 S-boxes in parallel, and the 4×4 S-box is implemented with one NOT gate, two AND, two OR and seven XOR gates. The control logic module of SCENERY manages control signals.

SCENERY encrypts a 64-bit plaintext using an 80-bit key, requiring 28 clock cycles. The hardware implementation of SCENERY requires 1438 GE. At a frequency of 100 KHz, sim-

ulated power consumption of SCENERY is $1.80\mu\text{W}$. Table 6 shows the implementation comparison between SCENERY and other ciphers.

The 64-bit plaintext requires approximately 384 GE and the 80-bit key requires 480 GE. The 32-bit XOR operation of AddRoundKey needs 85.44 GE. The cost of one S-box in SCENERY is 24 GE, and eight S-boxes require 192 GE. The MixColumns requires 128.16 GE, and the right data addition also requires 32-bit XOR operation, costing 85.44 GE. Two S-boxes and 5-bit constant XOR operations in the key schedule require 48 GE and 10.56 GE, respectively. In addition, the control logic and other counters require 24 GE. Therefore, an estimated area of a round-based SCENERY is 1437.6 GE, as shown in Table 7.

5.2 Software implementation

We present the implementation result of SCENERY on 64-bit processors and 8-bit micro-controllers.

1) 64-bit processors

We implemented SCENERY on a 2.3 GHz Intel(R) Core i5-8300H CPU running a 64-bit operating system with an Intel C++ compiler.

The S-boxes can be implemented using a sequence of 12 basic logical instructions. The MixColumns also needs 12 basic logical instructions. The key addition and the right data addition need four XOR operations, respectively. For a single block data, the software implementation of SCENERY gives a speed of approximately 45.5 cycles/byte. The software performance of SCENERY on 64-bit processors is relatively impressive from Table 8.

2) 8-bit microcontrollers

We evaluated the software performance on 8-bit microcontrollers through [25]. The SCENERY can be implemented on 8-bit microcontrollers through a series of logical operations. The SubColumns needs 14 instructions, and the MixColumns needs 17 instructions, as shown in Table 9. The subkey addition and the right data addition require four instructions, respectively. The implementation on 8-bit microcontrollers needs about 1516 clock cycles for a single block.

Table 6 Comparison of lightweight block cipher implementations

Ciphers	Structure	latency	Block size	Key size	Area/GE	Speed (kbps@100 KHz)	Logic process/um	Ref.
SFN	SPN	32	64	96	1877	200*	0.18	[1]
PRESENT	SPN	32	64	80	1570	200*	0.18	[8]
QTL	GFN	16	64	64	1026	200	0.18	[1]
RECTANGLE	SPN	25	64	80	1600	246	0.13	[2]
KLEIN	SPN	16	64	80	2202	400*	0.18	[7]
LBlock	Feistel	32	64	80	1320	200*	0.18	[6]
Twine	GFN	36	64	80	1503	178.78*	0.09	[5]
LED	SPN	32	64	80	1,040	3.4	0.18	[22]
Piccolo	GFN	25	64	80	1496	237.04	0.13	[2]
SIMON	Feistel	44	64	128	1751	145.45*	0.13	[23]
SPECK	Feistel	27	64	128	2014	237.04*	0.13	[23]
Midori	SPN	16	64	128	1542	400*	0.09	[1]
SKINNY	SPN	36	64	128	1696	177.78*	0.18	[24]
SCENERY	Feistel	28	64	80	1438	228.57*	0.18	This work

* denotes that the throughput at 100 MHz = $\frac{100 \text{ MHz} \times \text{block size}}{\text{latency}}$.

Table 7 Area requirement of SCENERY

Module (round function)	GE
Data Register	384
Key Xor	85.44
S-box layer	192
Mixcolumn	128.16
32-bit Xor	85.44
KS: Key Register	480
KS: SubCell	48
KS: Constants Xor	10.56
Control logic and other counters	24
Total	1437.6

Table 8 Comparison of software performance

Ciphers	Block size	Key size	One block enc.	Ref.
LED	64	64	65	[2]
Piccolo	64	80	67.1	[2]
PRESENT	64	80	62	[2]
RECTANGLE	64	80	30.5	[2]
TWINE	64	80	52.8	[5]
SIMON	64	128	28.7	[3]
SPECK	64	128	10.1	[3]
SCENERY	64	80	45.5	This work

“one block enc.” means the encryption of a single block.

Table 9 The Implementation of components on AVR

Components	Instruction	Flash/bytes	Time/cycles
SubColumns	MOV t_0, x_2	2	1
	EOR x_2, x_1	2	1
	COM x_1	2	1
	MOV t_1, x_0	2	1
	AND x_0, x_1	2	1
	OR x_1, x_3	2	1
	EOR x_3, t_0	2	1
	EOR x_0, x_3	2	1
	EOR x_1, t_1	2	1
	AND x_3, x_1	2	1
	EOR x_3, x_2	2	1
	OR x_2, x_0	2	1
	EOR x_2, t_1	2	1
	EOR x_1, t_0	2	1
MixColumns	MOVW t_0, x_1	2	1
	SWAP x_0	2	1
	EOR t_0, x_0	2	1
	LSL t_0	2	1
	ADC t_0, zero	2	1
	LSL x_0	2	1
	ADC x_0, zero	2	1
	EOR x_0, t_0	2	1
	EOR x_1, t_0	2	1
	SWAP x_3	2	1
	LSL t_1	2	1
	ADC t_1, zero	2	1
	EOR t_1, x_3	2	1
	EOR x_2, t_1	2	1
	ROL x_3	2	1
	ADC x_3, zero	2	1
	EOR x_3, t_1	2	1

Table 10 Comparison of software implementations for lightweight block ciphers on AVR

Ciphers	Block/Key size	Enc (cycles)	Ref.
SIMON	64/128	1969	[26]
SPECK	64/128	1141	[26]
SKINNY	64/128	2551	[26]
RECTANGLE	64/80	1823	[26]
SCENERY	64/80	1516	This work

“Enc” means the encryption process.

The evaluation of SCENERY on 8-bit microcontrollers was compared with other block ciphers, as shown in Table 10. From Table 10, we can conclude that the software performance of the encryption process is better than RECTANGLE and SIMON but worse than SPECK. Overall, we can conclude that SCENERY has a good performance on 8-bit microcontrollers.

6 Conclusion

This paper proposes SCENERY based on the bit-slice techniques. The bit-slice design principle allows for low-cost hardware and efficient software implementation. The algorithm adopts the binary matrix, and SCENERY can reach the full dependency after four rounds. The round function can be implemented using a sequence of 32 basic logical instructions. The hardware implementation of SCENERY with the 0.18 μm technology library requires 1438 GE, which is lower than the low-power RFID tags (2000 GE). Consequently, SCENERY has high security and high efficiency. The test vectors are given in Appendix.

Acknowledgements This research was supported by the Scientific Research Fund of Hunan Provincial Education Department (19A072), Application-oriented Special Disciplines, Double First-Class University Project of Hunan Province (Xiangjiaotong [2018] 469), the Science and Technology Plan Project of Hunan Province (2016TP1020).

Appendixes

Test vectors of SCENERY are given here. The data are shown in hexadecimal notation as follows.

plaintext:	0000-0000-0000-0000
key:	0000-0000-0000-0000
ciphertext:	82EF-EDBA-3336-CD92
plaintext:	0000-0000-0000-0000
key:	FFFF-FFFF-FFFF-FFFF
ciphertext:	CE6E-5005-CF04-E426
plaintext:	FFFF-FFFF-FFFF-FFFF
key:	0000-0000-0000-0000
ciphertext:	480B-5421-D561-1B60
plaintext:	FFFF-FFFF-FFFF-FFFF
key:	FFFF-FFFF-FFFF-FFFF
ciphertext:	F752-C84E-8412-4C59

References

1. Li L, Liu B, Zhou Y, Zou Y. SFN: A new lightweight block cipher. *Journal of Microprocessors Microsyst*, 2018, 60: 138–150
2. Zhang W, Bao Z, Lin D, Rijmen V, Yang B, Ingrid V. RECTANGLE: A bit-slice lightweight block cipher suitable for multiple platforms. *Journal of Science China (Information Sciences)*, 2015, 58(12): 89–103
3. Beaulieu R, Shors D, Smith J, Treatman-Clark S, Weeks B, Wingers L. The Simon And Speck Families of Lightweight Block Ciphers. *Journal*

- of IACR Cryptol. Eprint Arch, 2013, 2013: 404–449
4. Banik S, Pandey S K, Peyrin T, Sasaki Y, Sim S M, Todo Y. GIFT: A small present. In: Proceedings of International Conference on Cryptographic Hardware and Embedded Systems. 2017, 321–345
 5. Kobayashi E, Suzaki T, Minematsu K, Morioka S. TWINE: A lightweight block cipher for multiple platforms. In: Proceedings of the conference on Selected Areas in Cryptography. 2012, 339–354
 6. Wu W, Zhang L. LBlock: A lightweight block cipher. In: Proceedings of International Conference on Applied Cryptography and Network Security. 2011, 327–344
 7. Gong Z, Nikova S, Law Y W. KLEIN: A new family of lightweight block ciphers. In: Proceedings of Workshop on RFID Security. 2011, 1–18
 8. Bogdanov A, Knudsen L R, Leander G, Paar C, Poschmann A, Robshaw M J B, Seurin Y, Vikkelsoe C. PRESENT: An ultra lightweight block cipher. In: Proceedings of International Workshop on Cryptographic Hardware and Embedded Systems. 2007, 450–466
 9. Biham E. A fast new DES implementation in software. In: Proceedings of International Workshop on Fast Software Encryption. 1997, 260–270
 10. Saberi I, Shojaie B, Salleh M, Enhanced Key Expansion for AES-256 by using Even-Odd method. In: Proceedings of International Conference on Research and Innovation in Information Systems. 2011, 1–5
 11. Kam J B, Davida G I. Structured Design of Substitution-Permutation Encryption Networks. *Journal of IEEE Transactions on Computers*, 1979, C-28(10): 747–753
 12. Feistel H. Cryptography and Computer Privacy. *Journal of Scientific American - SCI AMER*, 1973, 228(5): 15–23
 13. Webster A F, Tavares S E, On the Design of S-Boxes. In: Proceedings of Lecture notes in computer sciences; 218 on Advances in cryptology—CRYPTO 85. 1986, 523–534
 14. Motara Y M, Irwin B. SHA-1 and the Strict Avalanche Criterion. In: Proceedings of Information Security for South Africa Conference. 2016, 35–40
 15. Knudsen L R. Practically secure Feistel ciphers. In: Proceedings of International Workshop on Fast Software Encryption. 1993, 211–221
 16. Mouha N, Wang Q, Gu D, Preneel B. Differential and Linear Cryptanalysis Using Mixed-Integer Linear Programming. In: Proceedings of International Conference on Information Security and Cryptology. 2011, 57–76
 17. Sun S, Hu L, Wang P, Qiao K, Ma X, Song L. Automatic Security Evaluation and (Related-key) Differential Characteristic Search: Application to SIMON, PRESENT, LBlock, DES(L) and Other Bit-oriented Block Ciphers. In: Proceedings of International Conference on the Theory and Application of Cryptology and Information Security. 2014, 158–178
 18. Liu B T, Li L, Wu R X, Xie M M, Li Q P. Loong: A family of Involutional lightweight Block Cipher Based on SPN Structure. *Journal of IEEE Access*, 2019, 7: 136023–136035
 19. Sasaki Y, Todo Y. New Impossible Differential Search Tool from Design and Cryptanalysis Aspects. In: Proceedings of Annual International Conference on the Theory and Applications of Cryptographic Technique. 2017, 185–215
 20. Biham E. New Types of Cryptanalytic Attacks Using Related Keys. *Journal of Cryptology*, 1994, 7(4): 229–246
 21. Biryukov A, Wagner D. Slide Attacks. In: Proceedings of International Workshop on Fast Software Encryption. 1999, 245–259
 22. Guo J, Peyrin T, Poschmann A, Robshaw M. The LED block cipher. In: Proceedings of International Workshop on Cryptographic Hardware and Embedded Systems. 2011, 326–341
 23. Beaulieu R, Shors D, Smith J, Treatman-Clark S, Weeks B, Wingers L. SIMON and SPECK: Block Ciphers for the Internet of Things. *Journal of IACR Cryptology ePrint Archive*. 2015, 2015: 585–599
 24. Beierle C, Jean J, Kölbl S, Leander G, Moradi A, Peyrin, T, Sasaki Y, Sasdrich P, Sim S M. The SKINNY Family of Block Ciphers and Its Low-Latency Variant MANTIS. In: Proceedings of Annual International Cryptology Conference. 2016, 123–153
 25. Dinu D, Biryukov A, Großschädl J, Khovratovich D, Corre Y L, Perrin L P. FELICS—Fair Evaluation of Lightweight Cryptographic Systems. In: Proceedings of NIST Workshop on Lightweight Cryptography, 2015
 26. Luo P, Zhang W, Bao Z. The Implementation and Optimization of Lightweight Block Cipher RECTANGLE based on FELICS. *Journal of Cyber Security*, 2017, 2(3): 36–47



Jingya Feng received the BS degree from Hebei University of Science and Technology, China in 2015. She has been working towards her Master's degree in Hunan Normal University, China since 2018. Her main research interest lies in embedded computing and information security.



Lang Li received the BS degree in circuits and systems from Hunan Normal University, China in 1996 and the master's and PhD degrees in computer science from Hunan University, China in 2006 and 2010, respectively. Since 2011, he has been a Professor with the College of Computer Science and Technology, Hengyang Normal University, China. His research interests include embedded computing and information security.